

BẢN DỊCH TIẾNG VIỆT

A tutorial on Particle Swarm Optimization Clustering

*Augusto Luis Ballardini
ballardini@disco.unimib.it
26 tháng 2 năm 2016*

Tài liệu này là bản dịch tiếng Việt phục vụ học tập từ file người dùng cung cấp. Một số thuật ngữ kỹ thuật được giữ kèm tiếng Anh trong ngoặc để dễ đối chiếu.

Tóm tắt

Bài viết này trình bày một hướng dẫn về kỹ thuật phân cụm dữ liệu sử dụng phương pháp Tối ưu hóa bầy đàn hạt (Particle Swarm Optimization - PSO). Dựa trên công trình của Merwe và cộng sự [1], tài liệu này cung cấp một phân tích chi tiết về thuật toán, đi kèm phần cài đặt bằng Matlab và một hướng dẫn ngắn giải thích cách chỉnh sửa phần cài đặt được đề xuất cũng như ảnh hưởng của các tham số trong thuật toán gốc. Ngoài ra, tài liệu còn so sánh kết quả thu được với phương pháp K-Means nổi tiếng. Toàn bộ mã nguồn được trình bày trong bài viết này được công khai theo giấy phép GPL-v2.

1. Giới thiệu nhẹ nhàng

Clustering là gì? Phân cụm có thể được xem là bài toán quan trọng nhất trong học không giám sát. Giống như các bài toán cùng loại, phân cụm xử lý việc tìm ra cấu trúc trong một tập dữ liệu chưa được gán nhãn [2]. Ta cũng có thể định nghĩa bài toán này là quá trình nhóm các vector dữ liệu đa chiều có tính tương đồng vào một số cụm, hay còn gọi là “bins” [1].

Dựa theo phương pháp được thuật toán sử dụng, có thể phân biệt bốn nhóm chuẩn: phân cụm loại trừ (Exclusive Clustering), phân cụm chồng lấn (Overlapping Clustering), phân cụm phân cấp (Hierarchical Clustering) và phân cụm xác suất (Probabilistic Clustering) [3]. Bên cạnh những kỹ thuật đã được thiết lập này, nhiều tác giả đã cố gắng khai thác Particle Swarm Optimization (PSO) [4] để phân cụm dữ liệu bất kỳ, ví dụ như dữ liệu ảnh.

Đóng góp của Merwe và Engelbrecht [1] đi theo đúng hướng này: họ trình bày hai cách tiếp cận dùng PSO để phân cụm dữ liệu, đồng thời đánh giá trên sáu bộ dữ liệu và so sánh với thuật toán phân cụm K-Means tiêu chuẩn.

Mặc dù người đọc có thể tìm thấy mô tả đầy đủ về các thuật toán được đề xuất trong bài báo gốc, trong phần còn lại của tài liệu này chúng ta sẽ thảo luận phần cài đặt Matlab của các thuật toán đó, kèm theo một số tay ngắn nhưng đầy đủ để sử dụng.

Cấu trúc tài liệu như sau: Mục 2 giới thiệu ngắn gọn kỹ thuật PSO và định nghĩa hình thức của nó. Mục 3 nêu bật lợi ích của PSO so với thuật toán K-Means hiện đại và phổ biến. Mục 4 trình bày các điểm chính trong mã Matlab, cung cấp những hiểu biết cần thiết để điều chỉnh mã cho các bộ dữ liệu hoặc nhu cầu phân cụm khác.

Hình 1: Ý tưởng đằng sau thuật toán PSO có thể được liên hệ với một đàn chim đang tìm kiếm thức ăn một cách ngẫu nhiên trong một khu vực.

2. Thuật toán PSO một cách ngắn gọn

Particle Swarm Optimization (PSO) là một phương pháp hữu ích để tối ưu hóa các hàm phi tuyến liên tục, mô phỏng các hành vi xã hội. Phương pháp này gắn với hiện tượng chim bay theo đàn, cá bơi theo đàn và nói chung là lý thuyết bầy đàn. Đây là một thuật toán rất hiệu quả nhưng vẫn đơn giản để tối ưu hóa nhiều loại hàm khác nhau [4].

Ý tưởng chính của thuật toán là duy trì một tập các lời giải tiềm năng, gọi là các particle. Mỗi particle đại diện cho một lời giải của bài toán tối ưu. Dựa trên hình ảnh đàn chim, một ví dụ trực quan được mô tả trong [5]: giả sử có một đàn chim đang tìm kiếm thức ăn trong một khu vực chỉ có một mẩu thức ăn. Tất cả các con chim không biết thức ăn ở đâu, nhưng tại mỗi bước thời gian chúng biết mình cách thức ăn bao xa. Chiến lược PSO dựa trên ý tưởng rằng cách tốt nhất để tìm thức ăn là đi theo con chim đang ở gần thức ăn nhất.

Quay lại bối cảnh phân cụm, ta có thể định nghĩa một lời giải là một tập các tọa độ, trong đó mỗi tọa độ tương ứng với vị trí c chiều của một tâm cụm. Trong bài toán PSO-Clustering, có thể tồn tại nhiều lời giải khả dĩ; mỗi lời giải gồm các vị trí tâm cụm c chiều. Cần chú ý rằng bản thân thuật toán có thể dùng trong không gian có số chiều bất kỳ, dù trong tài liệu này chỉ xét không gian 2D và 3D nhằm phục vụ trực quan hóa.

Mục tiêu của thuật toán là tìm giá trị đánh giá tốt nhất của một hàm fitness cho trước, hay trong trường hợp phân cụm là tìm cấu hình không gian tốt nhất của các tâm cụm. Vì mỗi particle đại diện cho một vị trí trong không gian Nd chiều, mục tiêu là điều chỉnh vị trí của particle dựa trên:

- vị trí tốt nhất mà chính particle đó tìm được cho đến hiện tại;
- vị trí tốt nhất trong vùng lân cận của particle đó.

Để thực hiện điều này, mỗi particle lưu các giá trị sau:

- x_i : vị trí hiện tại của particle;
- v_i : vận tốc hiện tại của particle;
- y_i : vị trí tốt nhất mà particle tìm được cho đến thời điểm hiện tại.

Sử dụng ký hiệu được giữ theo [1], vị trí của một particle được điều chỉnh theo:

$$v_{i,k}(t+1) = w v_{i,k}(t) + c_1 r_{1,k}(t)(y_{i,k}(t) - x_{i,k}(t)) + c_2 r_{2,k}(t)(\hat{y}_k(t) - x_{i,k}(t)) \quad (1)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (2)$$

Trong phương trình (1), w được gọi là trọng số quán tính (inertia weight), c_1 và c_2 là các hằng số gia tốc (acceleration constants), còn $r_{1,j}(t)$ và $r_{2,j}(t)$ được lấy mẫu từ phân phối đều $U(0,1)$. Vận tốc của particle được tính từ ba thành phần: (1) vận tốc trước đó, (2) thành phần nhận thức liên quan đến khoảng cách tới vị trí tốt nhất của chính particle, và (3) thành phần xã hội xét đến vị trí tốt nhất đạt được trong toàn bộ các particle của swarm.

Vị trí tốt nhất của một particle được tính bằng phương trình (3), tức là cập nhật vị trí tốt nhất nếu giá trị fitness tại bước thời gian hiện tại nhỏ hơn giá trị fitness tốt nhất trước đó của particle:

$$y_i(t+1) = y_i(t), \text{ nếu } f(x_i(t+1)) \geq f(y_i(t)); \text{ ngược lại } y_i(t+1) = x_i(t+1), \text{ nếu } f(x_i(t+1)) < f(y_i(t)) \quad (3)$$

PSO thường được thực thi bằng cách lặp liên tục phương trình (1) và (2) cho đến khi đạt số vòng lặp quy định. Một cách dừng khác là dừng khi vận tốc gần bằng 0, điều này cho thấy thuật toán đã đạt tới một cực tiểu trong quá trình tối ưu hóa.

Tài liệu [1] trình bày hai cách tiếp cận PSO là gbest và lbest. Trong gbest, thành phần xã hội được ràng buộc bởi toàn bộ swarm; trong lbest, thành phần xã hội được ràng buộc trong vùng lân cận hiện tại của particle. Tuy nhiên, tài liệu hướng dẫn này chỉ đề cập đến đề xuất gbest cơ bản.

Trước khi kết thúc mục này, cần giới thiệu cách đánh giá hiệu năng PSO tại mỗi bước thời gian, tức là một thước đo mô tả fitness của toàn bộ tập particle. Phương trình (4) hiện thực thước đo này, trong đó $|C_{i,j}|$ là số vector dữ liệu thuộc cụm C_{ij} , z_p là vector dữ liệu đầu vào thuộc cụm C_{ij} , m_j là tâm cụm thứ j của particle thứ i trong cụm C_{ij} , và N_c là số cụm:

$$J_e = [\sum_{j=1}^{N_c} (\sum_{\forall z \in C_{ij}} d(z_p, m_j) / |C_{i,j}|)] / N_c \quad (4)$$

Hình 2: Ví dụ sử dụng hai particle để phân cụm dữ liệu thành hai cụm trong không gian hai chiều. Mỗi particle được biểu diễn bằng một khung màu xanh lá; trong đó có thể thấy hai tâm cụm đang được theo dõi. Các chấm đen biểu diễn dữ liệu và giống nhau trong mỗi khung.

Hình 3: Bộ dữ liệu IRIS được khởi tạo với hai particle, trong hình là màu xanh lá và màu đỏ tím, mỗi particle sử dụng hai tâm cụm.

3. PSO so với K-Means

Trước khi chuyển sang phần mô tả mã, cần nhấn mạnh một số điểm quan trọng. Mục này tập trung làm rõ lợi ích của PSO so với thuật toán K-Means nổi tiếng. Mặc dù K-Means là một trong những thuật toán phân cụm phổ biến nhất, nhược điểm chính của nó là nhạy cảm với cách khởi tạo K tâm cụm ban đầu.

PSO giải quyết vấn đề này bằng cách kết hợp ba thành phần đã nêu ở Mục 2: quán tính, nhận thức và xã hội. Vì PSO là một phương pháp tìm kiếm dựa trên quần thể, nó giảm ảnh hưởng của điều kiện khởi tạo vì bắt đầu tìm kiếm đồng thời từ nhiều vị trí song song. Dù PSO thường hội tụ chậm hơn K-Means tiêu chuẩn, nó thường tạo ra kết quả chính xác hơn [6].

Một lưu ý cuối cùng là hiệu năng của PSO có thể được cải thiện thêm bằng cách khởi tạo swarm ban đầu bằng kết quả của K-Means. Ví dụ, dùng kết quả K-Means làm một particle, trong khi các particle còn lại được khởi tạo ngẫu nhiên. Cách này gọi là Hybrid-PSO và được mô tả rõ trong [1]; nó có thể cải thiện hiệu quả trong các cấu hình khó, như minh họa ở Hình 4.

Hình 4: Hai hình minh họa kết quả của thuật toán PSO khi có và không có gợi ý khởi tạo từ K-Means. Ở hình bên trái là Hybrid-PSO; ở hình bên phải là PSO tiêu chuẩn. Có thể thấy sự khác biệt nhỏ gần điểm dữ liệu có tọa độ (3,1), nơi điểm này bị gán nhãn sai ở hình bên phải nhưng được gán đúng trong hình bên trái. Dữ liệu trong hình lấy từ bộ IRIS, chỉ xét hai đặc trưng đầu tiên.

4. Giải thích mã nguồn

Mục này tập trung vào các điểm chính của thuật toán PSO thông qua phân tích các đoạn mã có ý nghĩa, đồng thời giải thích một số dòng Matlab có thể khó hiểu với người mới. Trong tất cả ví

dữ liệu của tài liệu, tác giả sử dụng bộ dữ liệu Fisher IRIS [7] có sẵn trong môi trường Matlab, đồng thời giảm số chiều xuống còn hai hoặc ba để dễ trực quan hóa dữ liệu và cụm. Các dòng mã trong các listing tương ứng với số dòng trong mã Matlab đã công bố.

Mã nguồn cung cấp các tham số sau:

```
29 centroids = 2
30 dimensions = 2
31 particles = 2
32 dataset_subset = 2
33 iterations = 50
34 simtime = 0.801
35 write_video = false
36 hybrid_pso = false
37 manual_init = false
```

Listing 1: Các tham số của thuật toán được đề xuất.

Trong danh sách trên, centroids biểu diễn số cụm mà người dùng muốn tìm, tức là số nhóm n chiều cần có trong dữ liệu đầu vào; giá trị này tương ứng với K trong thuật toán K-Means. Tham số dimensions xác định số chiều của mỗi tâm cụm, ví dụ trong không gian hai chiều thì đó là tọa độ x và y . Hai giá trị centroids và dimensions không phụ thuộc lẫn nhau; hoàn toàn có thể tìm hai cụm trong không gian ba chiều.

Tham số particles biểu diễn số swarm chạy song song tại cùng một thời điểm. Cần nhớ rằng mỗi swarm, cũng được gọi là particle, đại diện cho một lời giải hoàn chỉnh của bài toán. Trong trường hợp có hai tâm cụm trong không gian hai chiều, một particle là một cặp hai tọa độ dùng để xác định vị trí các tâm cụm. Tham số dataset_subset cho phép thay đổi kích thước bộ dữ liệu IRIS gốc bốn chiều của Matlab về giá trị chỉ định, nhằm trực quan hóa 2D hoặc 3D.

Các tham số còn lại tương đối dễ hiểu: iterations là số lần thuật toán được lặp lại trước khi dừng; simtime tạo độ trễ trực quan trong quá trình chạy script; write_video cho phép script ghi video từ các hình ảnh ở từng vòng lặp; hybrid_pso khởi tạo PSO bằng kết quả từ cài đặt K-Means tiêu chuẩn của Matlab; manual_init cho phép chỉ định vị trí ban đầu của các cụm nếu dimensions được đặt là 2.

Sau phần thiết lập môi trường ban đầu, mã định nghĩa ba biến w , $c1$, $c2$ tương ứng với các tham số trong phương trình (1), điều khiển đóng góp của quán tính, nhận thức và xã hội. Trong mã, các giá trị này được đặt theo [1, 8] để bảo đảm hội tụ tốt.

```
41 w = 0.72; % QUÁN TÍNH
42 c1 = 1.49; % NHẬN THỨC
43 c2 = 1.49; % XÃ HỘI
```

Listing 2: Các tham số cụ thể của thuật toán PSO.

4.1. Gán dữ liệu vào cụm

```
176 for particle=1:particles
177     [value, index] = min(distances(:, :, particle), [], 2);
178     c(:, particle) = index;
179 end
```

Listing 3: Một thao tác gán không hoàn toàn hiển nhiên trong Matlab.

Việc hiện thực phương trình (4) để tính fitness của một particle về mặt ý tưởng khá đơn giản, nhưng phần cài đặt Matlab có thể khó hiểu lúc đầu. Mã tính fitness bắt đầu ở dòng 218 bằng việc kiểm tra xem trong mảng c có ít nhất một phần tử thuộc tâm cụm đang xét hay không.

Fitness cục bộ sau đó được định nghĩa là trung bình của tất cả khoảng cách giữa các điểm thuộc về từng tâm cụm.

Do nhiều particle có thể được đánh giá song song, dòng 216 thêm một vòng lặp để mã có thể lặp qua nhiều lần đánh giá fitness của swarm. Trong vòng lặp thứ hai, các dòng 228 và 229 lưu và cập nhật hai giá trị bổ sung: fitness cục bộ tốt nhất và vị trí tốt nhất tìm được cho đến hiện tại. Sau đó, các dòng 233 và 234 trích xuất giá trị fitness tốt nhất và vị trí tốt nhất trong toàn bộ swarm hiện đang dùng. Tổng quan quá trình này được minh họa ở Hình 5(a).

```
216 for particle=1:particles
217     for centroid = 1 : centroids
218         if any(c(:,particle) == centroid)
219             local_fitness = ...
220                 mean(distances(c(:,particle)==centroid,centroid,particle));
221             average_fitness(particle,1)=average_fitness(particle,1)...
222                 + local_fitness;
223         end
224     end
225     average_fitness(particle,1) = average_fitness(particle,1) / ...
226         centroids;
227     if (average_fitness(particle,1) < swarm_fitness(particle))
228         swarm_fitness(particle) = average_fitness(particle,1);
229         swarm_best(:, :, particle) = swarm_pos(:, :, particle); % LOCAL BEST
230     end % FITNESS
231 end
232
233 [global_fitness, index] = min(swarm_fitness); % GLOBAL BEST FITNESS
234 swarm_overall_pose = swarm_pos(:, :, index); % GLOBAL BEST POSITION
```

Listing 4: Đoạn mã điều khiển quá trình đánh giá fitness. Cần chú ý rằng global fitness được đánh giá sau khi toàn bộ tập local fitness đã được đánh giá.

Phần cuối của mã liên quan đến việc cập nhật các thành phần quán tính, nhận thức và xã hội để thiết lập vận tốc của các particle. Ngoài thành phần quán tính chỉ được nhân với tham số w , các thành phần còn lại sử dụng các vị trí tốt nhất cục bộ và toàn cục đã tính trước đó, tương ứng với khía cạnh nhận thức và xã hội. Tổng của các thành phần này tạo thành swarm velocity, sau đó được dùng để cập nhật vị trí tổng thể của swarm. Ở các dòng 49 và 51, các biến $r1$, $r2$, $c1$ và $c2$ tương ứng với tham số trong phương trình (1).

```
47 for particle=1:particles
48     inertia = w * swarm_vel(:, :, particle);
49     cognitive = c1 * r1 * ...
50         (swarm_best(:, :, particle)-swarm_pos(:, :, particle));
51     social = c2 * r2 * (swarm_overall_pose-swarm_pos(:, :, particle));
52     vel = inertia+cognitive+social;
53     % CẬP NHẬT PARTICLE ...
54     swarm_pos(:, :, particle) = swarm_pos(:, :, particle) + vel ; % .. VỊ TRÍ
55     swarm_vel(:, :, particle) = vel; % .. VẬN TỐC
56 end
```

Listing 5: Đoạn mã cho thấy cách cập nhật vị trí của toàn bộ swarm.

4.2. Thay thế bộ dữ liệu IRIS

Mã được cung cấp được điều chỉnh cho bộ dữ liệu IRIS của Matlab với một cấu hình cụ thể, nghĩa là phần trực quan hóa chủ yếu hoạt động với đầu vào hai chiều và ba chiều. Điều này được thực hiện bằng cách giảm kích thước bộ dữ liệu IRIS gốc 150×4 xuống 150×2 hoặc 150×3 . Việc giảm chiều chỉ nhằm mục đích trực quan hóa vì chỉ dữ liệu hai hoặc ba chiều mới có thể hiển thị hiệu quả trên đồ thị.

Các thử nghiệm dựa trên bộ dữ liệu được cung cấp trong [9] cho thấy có thể sử dụng bộ dữ liệu nhiều chiều hơn, tức là nhiều hơn ba đặc trưng, với cả hai cách tiếp cận hiện có bằng cách thay đổi cơ bản ở các dòng 56 đến 58.

```
56 load fisheriris.mat
57 meas = meas(:,1+dataset_subset:dimensions+dataset_subset);
58 dataset_size = size(meas);
```

Listing 6: Cách tải bộ dữ liệu đầu vào.

4.3. Ghi video

Tác giả bổ sung một số dòng mã để cho phép ghi video dễ dàng. Tính năng này được thực hiện bằng các hàm Matlab `getframe` và `writeVideo`. Cách dùng đơn giản như sau: các dòng 47 đến 49 mở hệ thống tệp với tên tệp đầu ra là `PSO.avi`, được lưu trong cùng thư mục với mã Matlab. Các dòng 243 và 244 chụp và thêm một hình ảnh vào video, còn các dòng 295 đến 297 đóng tệp đã mở trước đó.

```
47 writerObj = VideoWriter('PSO.avi');
48 writerObj.Quality=100;
49 open(writerObj);
243 frame = getframe(fh);
244 writeVideo(writerObj,frame);
295 frame = getframe(fh);
296 writeVideo(writerObj,frame);
297 close(writerObj);
```

5. Kết luận

Trong tài liệu này, tác giả đã trình bày một giải thích có hệ thống về thuật toán PSO được đề xuất trong [1] thông qua việc phân tích mã nguồn công khai tại [10]. Mã nguồn cung cấp cả lựa chọn PSO tiêu chuẩn và Hybrid-PSO, cho phép người dùng nắm được từng chi tiết của công trình gốc. Mặc dù mã ban đầu được điều chỉnh để chạy với bộ dữ liệu IRIS của Matlab, nó có thể dễ dàng được chỉnh sửa để thực hiện phân cụm cho gần như bất kỳ loại bộ dữ liệu nào chỉ với những thay đổi mã tối thiểu.

Lời cảm ơn

Tác giả cảm ơn Tiến sĩ Axel Furlan vì sự hỗ trợ trong quá trình viết nguyên mẫu đầu tiên của mã Matlab sau đây.

6. Mã Matlab

Dưới đây là mã Matlab được dịch chú thích chính sang tiếng Việt. Các lệnh và tên biến được giữ nguyên hoặc chuẩn hóa bằng dấu gạch dưới để người đọc dễ đối chiếu với bản gốc.

```
1 % Tác giả: Augusto Luis Ballardini
2 % Email: augusto.ballardini@disco.unimib.it
3 % Website: http://www.ira.disco.unimib.it/people/ballardini-augusto-luis/
4
5 % Thư viện này được phân phối với hy vọng rằng nó sẽ hữu ích,
6 % nhưng KHÔNG CÓ BẤT KỲ BẢO HÀNH NÀO; kẻ` cả bảo đảm ngụ ý về`
7 % KHẢ NĂNG THƯƠNG MẠI hoặc PHÙ HỢP VỚI MỘT MỤC ĐÍCH CỤ THỂ.
8 % Được phép sao chép, phân phối và/hoặc chỉnh sửa tài liệu này
9 % theo các điều khoản của GNU Free Documentation License, Phiên bản 1.3
```

```

10 % hoặc bất kỳ phiên bản mới hơn nào do Free Software Foundation công bố.
11 % Không có phần bắt buộc, không có văn bản bìa trước và bìa sau.
12 % Một bản sao giấy phép được đưa vào phần "GNU Free Documentation License".
13
15 % Mã dưới đây được lấy cảm hứng từ bài báo:
16 % Van Der Merwe, D. W.; Engelbrecht, A. P., "Data clustering using particle
17 % swarm optimization," Evolutionary Computation, 2003. CEC '03.
18 % The 2003 Congress on, vol.1, pp.215-220 Vol.1, 8-12 Dec. 2003
19 % doi: 10.1109/CEC.2003.1299577
20 % URL:
21 % http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1299577
22
23 clear;
24 close all;
25
26 rng('default') % Để kết quả có thể tái lập
27
28 % KHỞI TẠO PARTICLE SWARM
29 centroids = 2; % số cụm ở đây, hay số centroid
30 dimensions = 2; % số chiều của mỗi centroid
31 particles = 1; % số particle trong swarm, tức số lời giải
32 iterations = 50; % số vòng lặp của thuật toán tối ưu
33 simtime=0.101; % độ trễ mô phỏng giữa các vòng lặp
34 dataset_subset = 2; % với bộ IRIS, thay đổi giá trị này từ 0 đến 2
35 write_video = false; % bật/đề ghi hình ảnh đầu ra thành video
36 hybrid_pso = true; % bật/tắt hybrid PSO
37 manual_init = false; % bật/tắt khởi tạo thủ công
38 % chỉ dùng cho dimensions = {2,3}
39
40 % THAM SỐ TOÀN CỤC (paper báo cáo các giá trị 0.72; 1.49; 1.49)
41 w = 0.72; % QUẢN TÍNH
42 c1 = 1.49; % NHẬN THỨC
43 c2 = 1.49; % XÃ HỘI
44
45 % PHÂN GHI VIDEO...
46 if write_video
47     writerObj = VideoWriter('PSO.avi');
48     writerObj.Quality=100;
49     open(writerObj);
50 end
51
52 % TẢI CỤM MẶC ĐỊNH (BỘ IRIS); CÂN DÙNG CÂN THẬN!
53 % Giảm kích thước dataset theo biến dimensions. Bộ iris chuẩn trong Matlab
54 % có kích thước 150x4; trong hướng dẫn này dùng 150x2 hoặc 150x3
55 % nhằm phục vụ trực quan hóa.
56 load fisheriris.mat
57 meas = meas(:,1+dataset_subset:dimensions+dataset_subset);
58 dataset_size = size(meas);
59
60 % Chạy k-means nếu bật hybrid pso. Nếu bật, vị trí khởi tạo
61 % của thuật toán pso sẽ được thiết lập bằng đầu ra của cài đặt k-means
62 % tiêu chuẩn trong Matlab.
63 if hybrid_pso
64     fprintf('Running Matlab K-Means Version\n');
65     [idx,KMEANS_CENTROIDS] = kmeans(meas, ...
66         centroids, ...
67         'dist', ...
68         'sqEuclidean', ...
69         'display', ...
70         'iter', ...
71         'start', ...
72         'uniform', ...
73         'onlinephase', ...
74         'off');

```

```

75     fprintf('\n');
76 end
77
78 % PHÂN VẼ ĐÔ THỊ: HANDLER VÀ MÀU SẮC
79 % Các dòng này cấu hình trước các biến dùng để vẽ dữ liệu.
80 pc = []; txt = [];
81 cluster_colors_vector = rand(particles, 3);
82
83
84 % VẼ DATASET
85 % Khởi tạo đồ thị 2D hoặc 3D tùy theo biến dimensions.
86 % Đề trực quan hóa, chỉ nên dùng giá trị 2 hoặc 3.
87 fh=figure(1);
88 hold on;
89 if dimensions == 3
90     plot3(meas(:,1),meas(:,2),meas(:,3),'k*');
91     view(3);
92 elseif dimensions == 2
93     plot(meas(:,1),meas(:,2),'k*');
94 end
95
96
97 % THIẾT LẬP TRỤC ĐÔ THỊ
98 % Nếu không có dòng này, giá trị max/min của trục có thể thay đổi khi chạy.
99 axis equal;
100 axis(reshape([min(meas)-2; max(meas)+2],1,[]));
101 hold off;
102
103
104 % THIẾT LẬP CẤU TRÚC DỮ LIỆU PSO
105 % Các biến cần cho pso clustering được khởi tạo trước.
106 % swarm_vel, swarm_pos và swarm_best lưu giá trị cho tất cả swarm/particle.
107 % c = ma trận lưu cụm được gán cho từng điểm.
108 % ranges dùng để co giãn giá trị ngẫu nhiên ban đầu vào khoảng dữ liệu đầu vào.
109 % swarm_fitness ban đầu là vô cực và sẽ giảm dần khi tối ưu fitness.
110 swarm_vel = rand(centroids,dimensions,particles)*0.1;
111 swarm_pos = rand(centroids,dimensions,particles);
112 swarm_best = zeros(centroids,dimensions);
113 c = zeros(dataset_size(1),particles);
114 ranges = max(meas)-min(meas);
115 swarm_pos = swarm_pos .* repmat(ranges,centroids,1,particles) + ...
116     repmat(min(meas),centroids,1,particles);
117 swarm_fitness(1:particles)=Inf;
118
119
120 % KHỞI TẠO BẰNG KMEANS
121 % Nếu chọn hybrid pso, thay swarm/lời giải đầu tiên bằng kết quả k-means.
122 % Dù dùng khởi tạo này, pso vẫn sẽ cố gắng cải thiện gợi ý ban đầu
123 % vì vận tốc của swarm vẫn được đặt ngẫu nhiên.
124 if hybrid_pso
125     swarm_pos(:, :, 1) = KMEANS_CENTROIDS;
126 end
127
128
129 % KHỞI TẠO THỦ CÔNG (chỉ cho dimension 2 và 3)
130 if manual_init
131     if dimensions == 3
132         swarm_pos(:, :, 1) = [6 3 4; 5 3 1];
133     elseif dimensions == 2
134         swarm_pos(:, :, 1) = ginput(2);
135     end
136 end
137
138
139 % Thuật toán PSO thực sự bắt đầu ở đây
140 for iteration=1:iterations
141
142     % TÍNH KHOẢNG CÁCH EUCLIDEAN TỚI TẤT CẢ CENTROID
143     distances=zeros(dataset_size(1),centroids,particles);
144     for particle=1:particles

```



```

159         for centroid=1:centroids
160             distance=zeros(dataset_size(1),1);
161             for data_vector=1:dataset_size(1)
162                 distance(data_vector,1) = norm( ...
163                     swarm_pos(centroid,:,particle)-meas(data_vector,:));
164             end
165             distances(:,centroid,particle) = distance;
166         end
167     end
168 end
169
170 % GÁN DỮ LIỆU VÀO CỤM
171 for particle=1:particles
172     [value, index] = min(distances(:,:,particle),[],2);
173     c(:,particle) = index;
174 end
175
176 % XÓA CÁC HANDLER ĐỒ THỊ TRƯỚC KHI VẼ LẠI
177 delete(pc); delete(txt);
178 pc = []; txt = [];
179
180 % VẼ LẠI TRẠNG THÁI HIỆN TẠI
181 hold on;
182 for particle=1:particles
183     for centroid=1:centroids
184         if any(c(:,particle) == centroid)
185             if dimensions == 3
186                 pc = [pc plot3(swarm_pos(centroid,1,particle), ...
187                     swarm_pos(centroid,2,particle), ...
188                     swarm_pos(centroid,3,particle),'*','color', ...
189                     cluster_colors_vector(particle,:))];
190             elseif dimensions == 2
191                 pc = [pc plot(swarm_pos(centroid,1,particle), ...
192                     swarm_pos(centroid,2,particle),'*','color',...
193                     cluster_colors_vector(particle,:))];
194             end
195         end
196     end
197 end
198
199 set(pc,{'MarkerSize'},{12})
200 set(gca,'LooseInset',get(gca,'TightInset'));
201 hold off;
202
203 % TÍNH GLOBAL FITNESS và LOCAL FITNESS = swarm_fitness
204 % Fitness được đo bằng quantization error theo phương trình 8 của paper gốc.
205 average_fitness = zeros(particles,1);
206 for particle=1:particles
207     for centroid = 1 : centroids
208         if any(c(:,particle) == centroid)
209             local_fitness = ...
210                 mean(distances(c(:,particle)==centroid,centroid,particle));
211             average_fitness(particle,1)=average_fitness(particle,1)...
212                 + local_fitness;
213         end
214     end
215     average_fitness(particle,1) = average_fitness(particle,1) / ...
216         centroids;
217     if (average_fitness(particle,1) < swarm_fitness(particle))
218         swarm_fitness(particle) = average_fitness(particle,1);
219         swarm_best(:,:,particle) = swarm_pos(:,:,particle); % LOCAL BEST
220     end
221 end
222
223 [global_fitness, index] = min(swarm_fitness); % GLOBAL BEST FITNESS
224 swarm_overall_pose = swarm_pos(:,:,index); % GLOBAL BEST POSITION
225
226

```

```

235 % IN THÔNG TIN RA COMMAND WINDOW
237 fprintf('%3d. global fitness is %5.4f\n',iteration,global_fitness);
238 pause(simtime);
239
240 % GHI VIDEO NẾU ĐƯỢC BẬT
242 if write_video
243     frame = getframe(fh);
244     writeVideo(writerObj,frame);
245 end
246
247 % LẤY MẪU r1 VÀ r2 TỪ PHÂN PHỐI ĐỀU [0..1]
250 r1 = rand;
251 r2 = rand;
252
253 % CẬP NHẬT CÁC CLUSTER CENTROID
257 for particle=1:particles
258     inertia = w * swarm_vel(:, :,particle);
259     cognitive = c1 * r1 * ...
260         (swarm_best(:, :,particle)-swarm_pos(:, :,particle));
261     social = c2 * r2 * (swarm_overall_pose-swarm_pos(:, :,particle));
262     vel = inertia+cognitive+social;
263
264     % CẬP NHẬT PARTICLE
265     swarm_pos(:, :,particle) = swarm_pos(:, :,particle) + vel ; % VI TRÍ
266     swarm_vel(:, :,particle) = vel; % VẬN TỐC
267 end
268
269 end % kết thúc thuật toán PSO
270
271 % VẼ QUAN HỆ GIỮA CÁC ĐIỂM VÀ CỤM
274 hold on;
275 particle=index; % chọn particle tốt nhất, tức fitness tốt nhất
276 cluster_colors = ['m','g','y','b','r','c','g'];
277 for centroid=1:centroids
278     if any(c(:,particle) == centroid)
279         if dimensions == 3
280             plot3(meas(c(:,particle)==centroid,1),meas(c(:,particle)== ...
281                 centroid,2),meas(c(:,particle)==centroid,3),'o','color',...
282                 cluster_colors(centroid));
283         elseif dimensions == 2
284             plot(meas(c(:,particle)==centroid,1), ...
285                 meas(c(:,particle)==centroid,2),'o','color', ...
286                 cluster_colors(centroid));
287         end
288     end
289 end
290 hold off;
291
292 % ĐÓNG FILE VIDEO NẾU ĐÃ MỞ
294 if write_video
295     frame = getframe(fh);
296     writeVideo(writerObj,frame);
297     close(writerObj);
298 end
299
300 % KẾT THÚC
301 fprintf('\nEnd, global fitness is %5.4f\n',global_fitness);

```

Tài liệu tham khảo

- [1] D. W. van der Merwe và A. P. Engelbrecht. Data clustering using particle swarm optimization. Evolutionary Computation, 2003. CEC '03, The 2003 Congress on, tập 1, trang 215-220, tháng 12 năm 2003.
- [2] Tagaram Soni Madhulatha. Comparison between K-Means and K-Medoids Clustering Algorithms, trong Advances in Computing and Information Technology, ACITY 2011, Springer Berlin Heidelberg, 2011.
- [3] Matteo Matteucci. A Tutorial on Clustering Algorithms.
- [4] J. Kennedy và R. Eberhart. Particle swarm optimization. IEEE International Conference on Neural Networks, tập 4, trang 1942-1948, 1995.
- [5] Xiaohui Hu. PSO Tutorial, 2006.
- [6] M. Omran, Ayed Salman và Andries P. Engelbrecht. Image classification using particle swarm optimization. Proceedings of the 4th Asia-Pacific Conference on Simulated Evolution and Learning, 2002.
- [7] Ronald A. Fisher. The use of multiple measurements in taxonomic problems. Annals of Eugenics, 7(2):179-188, 1936.
- [8] Frans Van Den Bergh. An analysis of particle swarm optimizers. Luận án tiến sĩ, University of Pretoria, 2006.
- [9] M. Lichman. UCI Machine Learning Repository, 2013.
- [10] Augusto Luis Ballardini. A Tutorial on Clustering Algorithms.
- [11] John A. Hartigan và Manchek A. Wong. Algorithm AS 136: A K-means clustering algorithm. Journal of the Royal Statistical Society, Series C, 28(1):100-108, 1979.

DIGITALLY SIGNED

By: Minh

Time: 2026-06-12 18:37:32

